

Programmation C++ moderne

Séance 3 – Classes, invariants et RAI

Stéphane Abide

Université Côte d’Azur – LJAD

Cours : 1 heure – TD/TP : 2 heures

Pourquoi protéger les données internes ?

Un état encore modifiable directement

```
IntArray values;  
  
values.size = 20;  
values.capacity = 8;
```

Comment utiliser 20 éléments avec seulement 8 places ?

Conséquence : l'objet peut devenir incohérent.

Besoin : contrôler les modifications.

Une mémoire encore gérée manuellement

```
values.data = new int[8];  
  
// ...  
  
delete[] values.data;
```

Qui garantit que `delete[]` sera appelé exactement une fois ?

Conséquence : oubli et double libération restent possibles.

Besoin : automatiser la libération.

Comment construire un objet qui protège son état et prend lui-même en charge sa mémoire ?

Les fonctions membres ne suffisent pas

Une interface prévue

```
struct IntArray {  
    int capacity{};  
    int size{};  
    int* data{nullptr};  
  
    void push_back(int value);  
    int at(int index) const;  
};
```

push_back peut vérifier la capacité, écrire la valeur puis mettre à jour la taille.

L'opération contrôlée conserve un état cohérent.

Mais les champs restent publics

```
IntArray values;  
  
values.push_back(3); // operation prevue  
  
// contourne push_back :  
values.size = -4;  
values.capacity = 2;  
values.data = nullptr;  
// compile
```

Ces écritures modifient directement la représentation et contournent toutes les vérifications de push_back.

L'objet peut redevenir incohérent.

Des fonctions membres correctes ne suffisent pas tant que les champs restent directement modifiables.

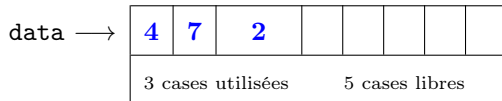
Question pour la suite : qu'est-ce qu'un IntArray correct ?

Quand IntArray est-il dans un état correct ?

Réponse à la question précédente

Un `IntArray` est correct lorsque ses trois champs décrivent ensemble un tableau utilisable.

capacity = 8 size = 3



Les trois règles

- ① $0 \leq \text{size} \leq \text{capacity}$;
- ② $\text{capacity} = 0 \Rightarrow \text{data} = \text{nullptr}$;
- ③ $\text{capacity} > 0 \Rightarrow \text{data}$ désigne `capacity` cases.

Les `size` premières cases contiennent les éléments utilisés ; les autres sont seulement disponibles.

État valide

capacity = 8
size = 3

État invalide

capacity = 8
size = 12
size > capacity

Invariant : une règle qui doit toujours rester vraie pour que l'objet soit correct.

Toute opération qui modifie `IntArray` doit le préserver.

L'état vide `size = 0`, `capacity = 0`, `data = nullptr` respecte lui aussi ces règles.

struct ou class ?

	struct	class
données	oui	oui
fonctions membres	oui	oui
accès par défaut	public	private

Données publiques simples

```
struct Point {  
    double x;  
    double y;  
};  
  
Point p{3.0, -2.0};
```

Les deux coordonnées sont directement accessibles : c'est le contrat choisi pour ce type simple.

Données liées par un invariant

size_ capacity_ data_



private interdit leur modification directe depuis l'extérieur



le code extérieur passe par l'interface publique

Une fonction membre mal écrite peut néanmoins encore casser l'invariant.

private ne garantit pas automatiquement les invariants.

Il empêche le code extérieur de contourner l'interface ; le constructeur et les fonctions membres doivent préserver les règles.

Les données deviennent privées

La représentation n'est plus accessible directement

```
class IntArray {  
private:  
    std::size_t capacity_{};  
    std::size_t size_{};  
    int* data_{nullptr};  
  
public:  
    void push_back(int value);  
};  
  
IntArray values;  
values.size_ = 100; // erreur : membre privé
```

Jusqu'ici, nous utilisons `int` pour simplifier. Désormais, tailles et indices utilisent `std::size_t`, comme la bibliothèque standard.

private crée une frontière de contrôle ; il ne prouve pas que la classe est correcte.

Le développeur de la classe doit écrire une interface qui préserve les invariants.

Depuis le code extérieur

× `size_` × `capacity_`
× `data_`



le compilateur impose le passage
par

✓ `push_back()`

✓ interface publique

Cette frontière réduit les endroits
où l'état peut être modifié.

Comment voulons-nous utiliser IntArray?

Le contrat public de la classe

```
class IntArray {  
public:  
    explicit IntArray(std::size_t capacity);  
    ~IntArray();  
  
    void push_back(int value);  
    int at(std::size_t index) const;  
    std::size_t size() const;  
};
```

Responsabilité de chaque opération

IntArray(...)	établir l'invariant
push_back()	modifier en le préservant
at() const	observer sans modifier
size() const	observer sans modifier
~IntArray()	libérer la ressource

Le constructeur établit l'invariant ; les opérations le préservent ; le destructeur libère la ressource.

La copie devra elle aussi produire un objet correct : ce problème sera laissé ouvert en fin de séance.

Un constructeur qui vérifie trop tard

Le code semble vérifier la capacité

```
IntArray::IntArray(std::size_t capacity)
: capacity_{capacity},
  size_{0},
  data_{new int[capacity]{} }
{
    if (capacity == 0) {
        throw std::invalid_argument{"zero capacity"};
    }
}
```

Mais l'initialisation vient avant le corps

- ① `capacity_ = capacity`
- ② `size_ = 0`
- ③ `new int[capacity]`
- ④ `tester capacity == 0`
- ⑤ `throw`

La ressource est demandée avant la validation.

Le constructeur est responsable de créer un objet correct.

Il doit vérifier les conditions avant d'acquérir la ressource.

La validation devient la première étape

```
std::size_t checked_capacity(std::size_t n) {  
    if (n == 0) {  
        throw std::invalid_argument{"zero capacity"};  
    }  
    return n;  
}  
  
IntArray::IntArray(std::size_t capacity)  
    : capacity_{checked_capacity(capacity)},  
      size_{0},  
      data_{new int[capacity_]}{}  
{}
```

Ordre obtenu

- ① vérifier capacity
- ② initialiser capacity_
- ③ initialiser size_
- ④ réserver data_

vérifier



allouer

Paramètre invalide : aucune ressource n'est acquise.

Paramètre valide : le constructeur établit l'invariant.

Choix d'interface : `IntArray{0}` est refusé, même si l'état interne `0 / 0 / nullptr` reste valide.

Éviter une conversion surprenante

Sans explicit : un appel ambigu pour le lecteur

```
class IntArray {  
public:  
    IntArray(std::size_t capacity);  
    std::size_t capacity() const;  
};  
  
void solve(IntArray values) {  
    std::cout << "capacity = " << values.capacity();  
}  
  
solve(20); // accepte : affiche capacity = 20
```

Ce que cherche le compilateur

1. Correspondance exacte ?

Non : aucune fonction `solve(int)`.

2. Conversion possible ?

Oui : le constructeur accepte l'entier.

$20 \longrightarrow \text{IntArray}\{20\} \longrightarrow \text{solve}(\dots)$

capacity = 20

Le compilateur construit un `IntArray`, mais l'appel `solve(20)` ne montre pas cette intention.

Éviter une conversion surprenante

Avec `explicit` : la construction doit être écrite

```
class IntArray {  
public:  
    explicit IntArray(std::size_t capacity);  
    std::size_t capacity() const;  
};  
  
void solve(IntArray values) {  
    std::cout << "capacity = " << values.capacity();  
}  
  
solve(20);           // erreur de compilation  
solve(IntArray{20}); // accepte : affiche capacity = 20
```

La recherche recommence

1. Correspondance exacte ?

Non : aucune fonction `solve(int)`.

2. Conversion possible ?

Non : `explicit` interdit la construction implicite.

`solve(20)` : refusé

`solve(IntArray{20})` : accepté

explicit oblige l'appelant à demander clairement la construction d'un `IntArray`.

Protéger les membres avec une méthode const

Le const final agit dans la méthode

```
std::size_t IntArray::size() const {  
    // size_ = 0; // erreur : modification interdite  
    return size_; // lecture autorisee  
}  
  
void IntArray::clear() {  
    size_ = 0; // autorise : methode non const  
}  
  
const IntArray values{8};  
values.size(); // oui : methode const  
values.clear(); // erreur : objet const
```

Deux vérifications du compilateur

size() const



l'objet courant est traité comme
constant

lire size_ oui

écrire size_ non

appeler clear() non

Un objet déclaré const ne donne
accès qu'aux méthodes elles-mêmes
déclarées const.

const fait vérifier une partie du contrat par le compilateur : la
méthode peut observer les membres, pas les modifier.

Le destructeur libère automatiquement la ressource

Le code utilisateur délimite une durée de vie

```
IntArray::~IntArray() {  
    delete[] data_;  
}  
  
void compute() {  
    IntArray samples{1000}; // construction  
  
    samples.push_back(4);  
    // ...  
} // fin de vie : destruction automatique
```

Aucun appel explicite à `delete[]` ni au destructeur dans `compute()`.

Ce que déclenche la portée

construction
↓
IntArray{1000}
acquisition de la ressource

↓
objet utilisable

↓
fin de la portée

↓
~IntArray()
delete[] data_

Le constructeur acquiert la ressource ; le destructeur la libère lorsque l'objet disparaît.

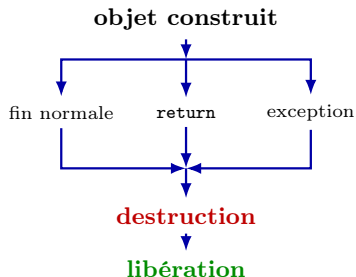
La gestion de cette ressource appartient désormais à la classe.

Et si on quitte la fonction plus tôt ?

Le code n'organise pas lui-même le nettoyage

```
double solve() {  
    IntArray workspace{1000};  
  
    if (!converged()) {  
        throw std::runtime_error{"no convergence"};  
    }  
  
    return residual(workspace);  
}
```

Toutes les sorties détruisent l'objet



La ressource suit la durée de vie de l'objet : c'est le RAII.
Quand l'objet disparaît, son destructeur libère sa ressource.

Dernier problème : que signifie copier IntArray ?

La copie générée recopie chaque champ

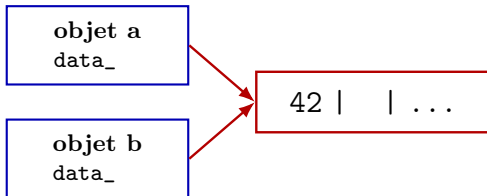
```
IntArray a{8};  
a.push_back(42);  
  
IntArray b{a};
```

capacity_ la valeur 8 est recopiée
size_ la valeur 1 est recopiée
data_ l'adresse est recopiée

```
IntArray(const IntArray&) = delete;  
IntArray& operator=(const IntArray&) = delete;
```

Solution temporaire : la copie est interdite.

Deux objets, une seule allocation



Modification partagée

Modifier le tableau par b change les données vues par a.

Double libération

Les deux destructeurs tenteront `delete[]` sur la même adresse.

Nous voulons qu'IntArray se comporte comme une valeur.
Après la copie : même contenu, mais ressources indépendantes.